

Міністерство освіти і науки України
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
“КИЄВО-МОГИЛЯНСЬКА АКАДЕМІЯ”
Кафедра мультимедійних систем факультету інформатики

ПРОМІЖНИЙ ЗВІТ

до курсової роботи за спеціальністю
“Інженерія програмного забезпечення”

Тема: Трасування променів у воксельному просторі

Виконавець: студент 3 курсу факультету інформатики, Олександр Гнутов
Керівник: старший викладач Борозенний С. О.

Київ 2026
31 січня 2026

Мета та завдання роботи

Метою курсової роботи є дослідити можливість застосування трасування променів у дискретному воксельному просторі та реалізувати демонстраційний застосунок, який виконує обчислення тіней та наближеного навколишнього перекриття у воксельному представленні сцени на GPU.

У межах поточного етапу сформульовано та деталізовано такі завдання:

1. Проаналізувати предметну область: воксельні структури даних, методи проходження променя у решітці (DDA та Amanatides–Woo), а також прийоми прискорення на GPU.
2. Побудувати прототип рендерера на базі рушія Bevy мовою Rust, що забезпечує завантаження воксельних даних та керування сценою.
3. Реалізувати загальний воксельний об'єм сцени (World Volume) у вигляді 3D текстури на GPU та сформувати MIP-рівні для пришвидшення пропуску порожнього простору.
4. Реалізувати постпроцесинговий відкладений (deferred) етап, що використовує дані з буфера глибини та нормалей (G-buffer) для запуску трасування у world volume.
5. Додати засоби для контролю параметрів і порівняння варіантів шейдера (наївний і оптимізований), а також виведення режимів візуалізації.

Короткий опис програмної реалізації

Проект реалізовано мовою Rust з використанням Bevy 0.15.3. Архітектурно застосунок побудований як набір плагінів, що інтегруються в Bevy App та RenderApp.

У файлах (src/lib.rs) і (src/main.rs) налаштовано запуск застосунку, камеру, базові об'єкти сцени та налагоджувальний інтерфейс. У застосунку передбачено інтерактивні параметри (через `egui`): вибір режиму візуалізації, величина зсуву (bias) для уникнення самозатінення, перемикання варіанту шейдера (naive або optimized), керування видимістю воксельних моделей та world volume.

Матеріали та буфери для відкладеного рендерингу

Для воксельних моделей використовується матеріал `VoxelRayTracedMaterial`, який зберігає 3D текстуру з воксельними даними. На етапі рендерингу матеріал

формує додаткові вихідні ресурси: буфер глибини та буфер нормалей, які далі використовуються постпроцесом.

У поточній версії сцена містить декілька прикладів воксельних об'єктів, зокрема дерево та рельєф, а також окремий об'єкт (умовна зброя), для якого передбачено режим ігнорування глибини.

World Volume як уніфікований простір сцени

Ключовим компонентом реалізації є World Volume, який представлено як 3D текстуру формату R8Uint. Такий формат дозволяє компактно зберігати бінарну інформацію про заповненість вокселя. Розмір world volume параметризується на рівні VoxelPlugin і в демонстраційній конфігурації встановлений як 256 на 256 на 256.

Заповнення world volume здійснюється обчислювальним шейдером. Для кожного воксельного об'єкта сцени на CPU формується матриця перетворення координат, яка переводить координати world volume до локальних координат моделі. У RenderApp виконується проходження по колекції активних воксельних об'єктів та запуск compute-проходу для внесення їх даних у загальний об'єм.

Для прискорення трасування у world volume реалізовано генерацію MIP-рівнів (у поточній конфігурації три рівні). Це дає змогу у шейдері використовувати крупніший крок проходження по решітці та швидше пропускати порожні ділянки, наближаючись до ідеї октодерева без явної побудови дерева.

Постпроцесингове трасування у воксельному просторі

Після основного проходу рендерингу запускається постпроцес DeferredRayTracedNode, який отримує:

1. екранну текстуру з базовим кольором
2. буфер глибини
3. буфер нормалей
4. world volume як 3D текстуру
5. додатковий шум або джитер для стохастичних вибірок

У WGSL реалізовано два варіанти фрагментного шейдера: базовий та оптимізований. Обидва варіанти використовують дані G-buffer для відновлення точки в просторі для кожного пікселя, обчислюють напрямок променя для вибірок (для тіней та АО) та виконують проходження по воксельній решітці.

Наївний варіант використовує фіксований MIP-рівень, що обирається через налаштування. Оптимізований варіант реалізує розріджене проходження `trace_sparse`, яке стартує з більш грубого рівня та динамічно зменшує MIP при наближенні до потенційного перетину, тим самим зменшуючи кількість кроків.

Також враховано проблему самозатінення. Для цього початкова точка променя зміщується вздовж нормалі поверхні та додатково вздовж напрямку променя на частку розміру вокселя.

Поточний стан виконання робіт

Станом на 31 січня 2026 виконано ключові пункти етапу прототипування:

1. Підготовлено робочий каркас застосунку на Bevy з базовою сценою, камерою та налагоджувальним інтерфейсом.
2. Реалізовано матеріали для воксельних моделей та формування допоміжних буферів (глибина і нормалі).
3. Реалізовано структуру `world volume` та `compute`-проходи для очищення, заповнення та генерації MIP-рівнів.
4. Реалізовано `deferred` постпроцес, який виконує трасування у `world volume` та моделює тінь і наближене `ambient occlusion`.
5. Додано можливість перемикання між двома шейдерами і декількома режимами візуалізації для перевірки коректності.

Проміжні результати та спостереження

Отримано працездатний прототип, який демонструє наступне:

1. Єдиний глобальний об'єм дозволяє уніфікувати трасування для множини об'єктів без побудови індивідуальних прискорювальних структур.
2. MIP-рівні дають практичний шлях до прискорення: у типових сценах з великою кількістю порожнього простору оптимізований прохід зменшує кількість звернень до `textureLoad`.
3. У воксельному представленні помітні артефакти самозатінення, тому `bias` та комбінований зсув (по нормалі й по променю) є обов'язковими для прийнятної якості.

4. Стохастичні вибірки для АО потребують компромісу між кількістю семплів і швидкістю. У поточній конфігурації використано невелику кількість вибірок, що дає шум, але зберігає продуктивність.

Проблеми та ризики

На поточному етапі ідентифіковано такі технічні ризики:

1. Обмеження за розмірами world volume та вартість пам'яті GPU. Зі збільшенням роздільності об'єму зростають витрати на зберігання та на генерацію MIP-рівнів.
2. Баланс якості й швидкодії. Для стабільного FPS потрібне коректне обмеження кількості кроків трасування та вибірок АО.
3. Відтворюваність якості при зміні масштабу сцени. При інших розмірах вокселя потрібна повторна калибровка bias та параметрів освітлення.

План робіт на наступний етап

До наступного контрольного етапу планується:

1. Реалізувати додаткові оптимізації та покращення (однобітний воксельний об'єм, тіньове акне при русі об'єктів, додаткові джерела світла).
2. Оформити розділ 2 текстової частини з описом DDA, Amanatides–Woo та логіки розрідженого обходу на основі MIP-рівнів.
3. Оформити розділ 3 з описом архітектури застосунку: модулі Rust, граф рендерингу Bevy, формування world volume та взаємодія з WGSL.
4. Підготувати набір візуальних результатів для демонстрації: режими нормалей, глибини, напрямку променя, а також порівняння наївного та оптимізованого варіанту.
5. Додати прості вимірювання продуктивності для різних розмірів world volume та різних режимів трасування, щоб сформулювати висновки щодо ефективності оптимізацій.

Висновок

На момент проміжного контролю реалізовано працюючий прототип трасування променів у воксельному просторі на GPU з використанням загального воксельного об'єму та двох варіантів шейдерної реалізації. Архітектура на базі Bevy дозволяє інтегрувати compute-етапи для world volume та фрагментний deferred-прохід у рендер-граф, а налагоджувальний інтерфейс дає змогу оперативно перевіряти коректність та параметри якості.

Література

- [1] Воксель. Вікіпедія. <https://uk.wikipedia.org/wiki/%D0%92%D0%BE%D0%BA%D1%81%D0%B5%D0%BB%D1%8C>
- [2] Laine S., Karras T. Efficient sparse voxel octrees. Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games, 2010.
- [3] Shirley P. Ray Tracing in One Weekend. <https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- [4] Yagel R., Cohen D., Kaufman A. Discrete ray tracing. IEEE Computer Graphics and Applications, 1992.